UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DE OCCIDENTE

DIVISIÓN DE CIENCIAS DE LA INGENIERÍA

INGENIERÍA EN CIENCIAS Y SISTEMAS

Manual técnico

REDXela · Centro de distribución concurrente

Proyecto Final — Sistemas Operativos 2
Segundo semestre de 2026

Arquitectura modular de Java 21, sincronización, comunicación entre áreas, almacenamiento y persistencia

Quetzaltenango, octubre de 2026

1. Objetivo y alcance

Este manual describe la reconstrucción modular del backend de REDXela. La interfaz Angular conserva el mismo diseño y el mismo contrato HTTP/JSON; las responsabilidades internas fueron separadas en 28 archivos Java para hacer visibles los mecanismos de Sistemas Operativos 2 y reducir el acoplamiento de la lógica.

La simulación procesa pedidos de clientes con cinco niveles de servicio, administra seis tipos físicos de recursos limitados, registra inventario en ubicaciones no contiguas, mantiene una recepción independiente de inventario, permite comparar cuatro políticas y presenta una situación de interbloqueo resuelta por decisión del usuario.

Alcance de la entrega

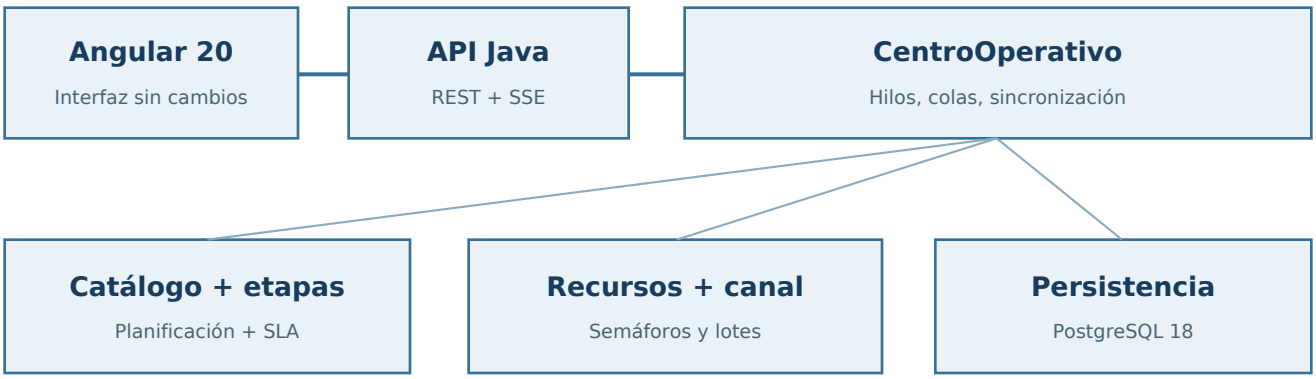
• Código fuente Java 21, código Angular 20, esquema SQL, Docker Compose y manuales PDF.

• Sin dependencias descargadas, proyectos duplicados, carpetas Git, compilados ni archivos de pruebas.

• Recurso virtual CONTROL_CONFLICTO de una unidad, usado solamente para representar claramente la espera circular; no sustituye ninguno de los recursos físicos exigidos.

2. Arquitectura y relaciones entre módulos

Se emplea una arquitectura de capas simple sin frameworks adicionales en el backend: un servidor HTTP nativo, un motor de simulación, colaboradores de negocio y adaptadores de infraestructura. Las clases quedan en el paquete por defecto y se compilan juntas mediante javac src/*.java para mantener el despliegue sencillo.



Flujo de una petición

Un clic en Angular genera una petición HTTP a EnrutadorApi. La ruta valida argumentos, invoca CentroOperativo y devuelve una respuesta serializada por RespuestaApi / ConversorJson. Las transiciones de la simulación ocurren bajo el monitor gate. La vista se actualiza con EventosOperativos, que emite Server-Sent Events cada 300 ms.

3. Trazabilidad de los requisitos del proyecto

Requisito	Implementación / evidencia técnica
Hilos Java y sincronización	CentroOperativo usa ExecutorService de 16 hilos para pedidos, dos para Recepción e Inventario y dos temporizadores. El estado se protege con synchronized(gate).
Recursos limitados	AdministradorRecursos emplea Semaphore(capacity, true), mantiene propietarios, ocupación y disponibilidad.
Cinco mercancías	CatalogoMercancia incluye GENERAL, FRAGIL, PESADO, VOLUMINOSO, ELECTRONICO.
Niveles y políticas	PoliticasAtencion: PRIORIDAD, FIFO, UNIDADES y ENVEJECIMIENTO. MedidorNivelServicio evalúa SLA.
Recepción e Inventario	CanalInventario mantiene una ArrayBlockingQueue de ocho avisos; son áreas independientes.
Almacenamiento físico	InventarioEspacial: 60 celdas de capacidad 10, lotes, ubicaciones y reservas.
Deadlock demostrable	Dos hilos retienen recursos en orden inverso; resolución manual mediante interrupción del perdedor.
Interfaz en tiempo real	EnrutadorApi devuelve snapshots JSON y EventosOperativos sirve /api/events.
Persistencia	RepositorioPostgresql guarda payload serializado y proyección JSONB en una transacción.

4. Organización de clases: presentación y dominio

Clase	Responsabilidad funcional
InicioServidor	Configurar Postgres o memoria, abrir servidor HTTP, iniciar hilos y cierre cooperativo.
EnrutadorApi	Validar rutas, métodos y parámetros de las operaciones sin mezclar lógica de inventario.
ParametrosApi / RespuestaApi	Decodificar query strings y serializar respuestas JSON y errores.
EventosOperativos	Transmitir actualizaciones en tiempo real mediante SSE.
CentroOperativo	Coordinar el ciclo de vida de pedidos, ingreso, inventario, despacho y conflicto.
PedidoOperativo / EtapaOperacion	Conservar estados, cantidades, tiempos, reservas y necesidades por etapa.
CuentaCliente / ProductoCatalogo / LineaPedido	Representar cuentas, productos fijos y líneas del pedido.
EntradaStock / AvisoInventario / EstadoGuardado	Representar recepciones, avisos asíncronos y checkpoint completo.

5. Organización de clases: servicios e infraestructura

Clase	Responsabilidad funcional
CatalogoMercancia	Contiene 10 productos y asignaciones de recursos de 5 tipos de mercancía.
ConstructorEtapas	Construye preparación, empaque opcional, calidad opcional, escaneo y carga.
AdministradorRecursos / RecursoOperativo	Semáforos, propietarios, semáforo FIFO, toma y liberación, estadísticas.
PoliticasAtencion	Ranking, envejecimiento, límites y evaluación de nivel de servicio.
AnalizadorPoliticas	Simulación de eventos discretos sobre la misma carga de pedidos.
MedidorNivelServicio / VistaPedido	Calcular indicadores y convertir pedidos al contrato JSON de Angular.
CanalInventario	Canal bloqueante acotado entre productor y consumidor.
InventarioEspacial	Controlar celdas, lotes, cantidades disponibles y reservas.
AlmacenEstado / RepositorioPostgresql / RepositorioVolatil	Abstracción de almacenamiento y alternativas PostgreSQL y memoria.
ConversorJson	Codificador JSON para mapas, listas y valores simples.

6. Recursos del centro de distribución

Los recursos existen como objetos RecursoOperativo con un Semaphore justo (FIFO de hilos que compiten por permisos), un arreglo de propietarios por unidad y marcas de tiempo para calcular la ocupación. Los permisos se adquieren solamente cuando la etapa puede empezar.

Clave	Capacidad	Uso
BODEGA	10	Encargados de bodega / preparación / traslado.
MONTACARGAS	3	Mercancía pesada y voluminosa.
EMPAQUE	6	Estaciones de empaque para artículos que lo necesitan.
CALIDAD	4	Inspección de mercancía frágil y electrónica.
ESCANER	1	Compartido entre ingreso de mercancía y despacho.
CARGA	2	Áreas de carga para despachar pedidos.
CONTROL_CONFLICTO	1	Recurso auxiliar exclusivamente demostrativo.

Evitar interbloqueo en el procesamiento regular

En CentroOperativo.tick() las etapas se ordenan conforme a la política activa. El motor comprueba que todos los permisos de una etapa estén disponibles, y bajo synchronized(gate) adquiere el conjunto. Una etapa no espera indefinidamente reteniendo un subconjunto. Si no hay permisos suficientes, el pedido permanece en EN_ESPERA y expone una razón visible. AdministradorRecursos.liberar() devuelve los permisos en un bloque finally al finalizar la etapa.

Trabajo proporcional a la cantidad de unidades

Los tiempos base de cada etapa se multiplican por el número total de artículos. El tiempo real de demostración se divide por SIMULATION_TIME_SCALE para poder observar fácilmente procesos largos. El indicador de servicio usa por separado clockRate como factor de conversión a tiempo simulado.

7. Flujo de pedidos y niveles de servicio

Nivel	Nombre	Espera objetivo
1	Exprés	Casi inmediata (umbral técnico de 1 min)
2	Prioritario	Hasta 10 min
3	Estándar	Hasta 30 min
4	Programado	Hasta 60 min
5	Económico	Hasta 120 min

El cliente se registra con correo único y selecciona su servicio. El pedido almacena el cliente, cada línea, el nivel contratado, las unidades, el estado, la etapa, el tiempo de espera, el tiempo de proceso y la reserva de inventario.

Políticas disponibles

• PRIORIDAD: nivel de servicio menor primero; identificador como desempate.

• FIFO: pedidos con menor identificador primero.

• UNIDADES: menor número de artículos primero, para liberar recursos con rapidez.

• ENVEJECIMIENTO: mejora de prioridad según el tiempo acumulado en espera y agingSeconds.

El criterio se implementa en PoliticasAtencion.rank(). Para evitar starvation bajo envejecimiento, cada intervalo de espera baja el nivel efectivo hasta un mínimo de 1. La política actual puede modificarse en la interfaz y su efecto se refleja inmediatamente en la cola.

Etapas de un pedido

ConstructorEtapas produce PREPARACION (150 ms/unidad), EMPAQUE (250 ms/unidad si aplica), CALIDAD (180 ms/unidad si aplica), DESPACHO_ESCANEO (40 ms/unidad) y DESPACHO_CARGA (80 ms/unidad). Se asignan distintas combinaciones de recursos por etapa. El escáner se libera antes de utilizar las áreas de carga.

8. Tipos de mercancía y sus recursos

Mercancía	Requisitos principales	Justificación
GENERAL	BODEGA, EMPAQUE	Liviano y con embalaje regular.
FRAGIL	BODEGA, EMPAQUE, CALIDAD	Necesita protección y evaluación.
PESADO	BODEGA, MONTACARGAS, CARGA	Movimiento mecánico y carga.
VOLUMINOSO	BODEGA, MONTACARGAS, EMPAQUE, CARGA	Volumen elevado y embalaje.
ELECTRONICO	BODEGA, CALIDAD, ESCANER	Inspección y registro de identificadores.

El conjunto de necesidades define qué etapas se agregan a cada pedido. La secuencia siempre concluye con escaneo de despacho y carga, por lo cual hay contención observable con el escáner compartido de Recepción, aun cuando la mercancía no sea electrónica.

9. Comunicación entre áreas: productor-consumidor

Recepción y Control de Inventario se ejecutan en dos hilos independientes de un pool. Se comunican con objetos AvisoInventario dentro de un ArrayBlockingQueue(8) administrado por CanalInventario. La recepción registra primero el documento de ingreso, solicita el escáner compartido y solo después encola la notificación.

Cuando el canal está lleno, el registro permanece con estado ESPERANDO_CANAL y vuelve a intentarse más adelante; ningún ingreso se descarta por saturación. El consumidor toma un aviso, simula la preparación, registra un lote y asigna espacios en el almacén. La propiedad lotId impide cargar dos veces un mismo ingreso.

Esquema del flujo

```
RECEPCION -> PENDIENTE_ESCANEO -> ESCANEANDO
          -> ESPERANDO_CANAL -> EN_COLA_INVENTARIO
          -> INVENTARIO_PROCESANDO -> REGISTRADA
                                   -> ESPERA_ESPACIO (reintento)
```

El escáner de entrada y el escáner de despacho son el mismo Semaphore de capacidad 1. La comunicación se realiza por mensajes y no por escritura directa de estructuras del otro proceso. Los pedidos en ESPERA_STOCK se reevalúan en tick() después de cada ingreso.

10. Organización del almacén

InventarioEspacial define 60 ubicaciones, cada una con capacidad de 10 unidades: el límite físico total es de 600 unidades. Las ubicaciones usan etiquetas de pasillo, nivel y posición (por ejemplo, A-N1-U1). La asignación visita las ubicaciones mediante el paso (posición × 17) mod 60, que cubre todas las celdas al ser 17 coprimo con 60. Así se representan lotes no contiguos.

Un lote contiene producto, unidades recibidas e identificador de recepción. Cada asignación contiene lote, ubicación, cantidad y cantidad reservada. El stock disponible es stock físico menos stock reservado. Los pedidos separan existencias antes de ejecutar etapas; el consumo real ocurre al completar el despacho. El ingreso se pospone si el nuevo lote no cabe por completo, evitando asignaciones parciales.

Propiedades de consistencia

• No se aceptan recepciones o pedidos con cantidades fuera de los rangos establecidos.

• No se asigna stock a dos pedidos distintos si solo existe una cantidad disponible.

• Las reservas se consumen al finalizar, y los lotes conservan sus asignaciones visibles.

• La simulación conserva pedidos sin stock en ESPERA_STOCK sin perder su posición lógica.

11. Interbloqueo demostrativo y recuperación manual

La ejecución normal evita la retención parcial. Para demostrar el problema pedido por el enunciado, CentroOperativo.startConflictInternal() inicia dos hilos Java: A adquiere ESCANER y luego solicita CONTROL_CONFLICTO; B toma CONTROL_CONFLICTO y luego solicita ESCANER. CountDownLatch(2) garantiza que ambos adquieran el primer recurso antes de pedir el segundo.

Cuando los dos hilos alcanzan la segunda adquisición sin conseguirla se marca INTERBLOQUEO. La vista muestra IDs de pedidos, recursos retenidos, recursos solicitados, motivos de bloqueo y bitácora del conflicto. Enviar un ganador a /api/conflict/resolve interrumpe al hilo perdedor, que libera en finally su recurso. El ganador puede continuar y ambos pedidos vuelven a la planificación normal con reservas conservadas.

Por qué este caso es un deadlock real

• Exclusión mutua: tanto ESCANER como CONTROL_CONFLICTO tienen una unidad.

• Retención y espera: cada hilo conserva el primer permiso mientras solicita el segundo.

• Sin expropiación espontánea: la liberación ocurre únicamente cuando se interrumpe el perdedor por decisión manual.

• Espera circular: el hilo A necesita el permiso de B y B necesita el de A.

CONTROL_CONFLICTO es un recurso artificial claramente identificado para garantizar repetibilidad sin interferir con las seis capacidades físicas exigidas. Después de liberar el bloqueo, los pedidos se procesan con las reglas normales de almacén y despacho.

12. Comparación de políticas e indicadores

AnalizadorPoliticas genera una carga reproducible de 10 a 100 pedidos con una semilla y las mismas combinaciones de recursos y etapas de la operación regular. Ejecuta cuatro simulaciones de eventos discretos independientes, una por política. Presenta duración total, pedidos terminados, promedio y máximo de espera, incumplimientos de SLA y ocupación por recurso.

Este experimento no inserta los pedidos comparativos en el almacén real; calcula tiempos con un calendario virtual y asume stock suficiente. Así se puede observar la diferencia entre planificadores sin cambiar existencias ni reservas de la ejecución concurrente.

MedidorNivelServicio calcula, por nivel, cantidad de pedidos medidos, completados, dentro y fuera de plazo, vencidos pendientes y media de espera. VistaPedido mantiene estos campos disponibles para el panel de Angular: waitMs, virtualWaitMs, slaLimitMs, slaStatus, effectiveLevel y nextPromotionMs.

13. Persistencia y recuperación

AlmacenEstado abstrae el almacenamiento. RepositorioPostgresql utiliza JDBC sobre PostgreSQL 18 y ejecuta una escritura transaccional con el checkpoint binario (BYTEA) y una proyección JSONB para consulta humana. El esquema de backend/db/V1__schema.sql crea la tabla centro_operativo_estado y vistas para clientes, pedidos, existencias, lotes, ubicaciones y recepciones.

El constructor JDBC adquiere un bloqueo asesor de PostgreSQL para rechazar una segunda instancia que intente modificar el mismo estado. Cuando hay un fallo de persistencia el motor no continúa planificando silenciosamente; informa el error y requiere recuperar conexión. Al reiniciar, restore() rehidrata pedidos y recepciones, conserva las reservas y devuelve operaciones incompletas a las colas, sin asumir que los hilos antiguos siguen vivos.

Compatibilidad con la versión anterior

Importante: los archivos binarios Java serializados con los nombres de clases anteriores NO se pueden leer automáticamente con las clases nuevas. Este entregable conserva las rutas HTTP y la apariencia Angular, pero la persistencia debe iniciarse en un volumen PostgreSQL nuevo. No elimine el volumen antiguo si desea conservar esos datos: primero realice un respaldo. La ejecución nueva usa un estado propio e independiente.

14. API y actualización en tiempo real

Ruta HTTP	Función
GET /api/health	Estado UP y nombre del sistema.
GET /api/state	Fotografía JSON completa de recursos, pedidos, clientes, almacén, SLA y conflicto.
GET /api/events	Stream SSE; envía un snapshot cada 300 milisegundos.
POST /api/clients?name=&email;=&level;=	Registrar cuenta de cliente.
POST /api/clients/service?clientId=&level;=	Actualizar servicio contratado.
POST /api/orders?clientId=&items;=1:5,2:3	Registrar pedido manual con líneas y unidades.
POST /api/receipts?productId=&units;=	Informar nueva entrada de productos.
POST /api/generator?enabled=true|false	Iniciar / detener generación automática.
POST /api/policy?value=	Seleccionar PRIORIDAD, FIFO, UNIDADES o ENVEJECIMIENTO.
POST /api/aging?seconds= /api/clock?rate=	Cambiar intervalo de envejecimiento / reloj virtual.
POST /api/comparison?seed=&count;=	Evaluar las cuatro políticas con igual carga.
POST /api/conflict/start?clientId=&productId;=&units;=	Provocar contención circular reproducible.
POST /api/conflict/resolve?winner=	Elegir el pedido que continúa.

Todas las operaciones de escritura mantienen el convenio de parámetros URL usado por la interfaz original. EnrutadorApi retorna JSON y mensajes de error con códigos 400 para parámetros inválidos, 404 para rutas inexistentes y 405 para métodos no permitidos.

15. Infraestructura, instalación y operación

Servicio	Imagen / tecnología	Acceso local
db	postgres:18-alpine; volumen persistente	Puerto 5439 → 5432
backend	eclipse-temurin:21-jdk; javac sin Maven	Puerto 8090
frontend	Angular 20, Node 22, Nginx	http://localhost:4200

Iniciar desde Windows 10/11 con Docker Desktop

```
cd RUTA\REDXela_Reconstruido
Copy-Item .env.example .env
docker compose up --build -d
docker compose ps
```

Abra http://localhost:4200. La API también puede consultarse en http://localhost:8090/api/health. El servicio backend espera a que el healthcheck de la base devuelva healthy. Para terminar conservando existencias y pedidos use docker compose down; nunca agregue -v salvo que desee eliminar intencionadamente datos del volumen de esta nueva instalación.

Compilar en Java 21 sin contenedor

```
cd backend
javac -encoding UTF-8 -d out src/*.java
# Windows con PostgreSQL disponible:
java -cp "out;lib/*" InicioServidor
```

Variables admitidas: DATABASE_URL, DATABASE_USER, DATABASE_PASSWORD, PORT, SIMULATION_TIME_SCALE y REDXELA_MODE. El modo memory sirve para una demostración temporal sin Postgres, pero no guarda datos después de cerrar.

16. Guion de demostración académica

Para enseñar los requisitos con la misma interfaz del sistema original: registrar un cliente; ingresar mercadería; observar el uso del escáner, canal e inventario; registrar pedidos de distintos niveles; cambiar de política y mostrar la cola y métricas; abrir Recursos para ver titulares/semáforos; usar Comparación con semilla fija; finalmente provocar INTERBLOQUEO y elegir el ganador en pantalla.

En el código, los mecanismos principales pueden localizarse en CentroOperativo.tick(), AdministradorRecursos.asignar()/liberar(), CentroOperativo.receptionLoop()/inventoryLoop(), CanalInventario, InventarioEspacial.reserve()/consume() y CentroOperativo.conflictRun()/resolveConflict().

Decisiones y limitaciones explícitas

• Se preservó Angular de forma íntegra para no introducir cambios visuales ni romper flujos ya utilizados.

• La cantidad de workers de pedidos es fija en 16 para evitar crear un hilo sin límite por pedido; la cola puede crecer conforme la demanda del generador.

• La simulación usa tiempos acelerados, por lo que representa comportamiento concurrente, no rendimiento real de un centro logístico.

• El caso de interbloqueo usa un recurso auxiliar intencional para hacerlo repetible y comprensible.

• El servidor HTTP es una API demostrativa local; no incluye autenticación ni controles de acceso para despliegue público.

• Los checkpoints binarios de la versión original requieren migración específica si se desea conservar el historial anterior.

Conclusión: la separación de capas y de entidades deja visibles las estructuras de concurrencia y almacenamiento, mejora la legibilidad y permite explicar las decisiones de Sistemas Operativos 2, conservando la experiencia de uso ya implementada.